THE SIDEBAR IS A SECURITY BOUNDARY
UI-AS-POLICY: UNIFIED PERMISSION AND LAYOUT SYSTEM FOR MULTI-TENANT PLATFORMS

SHAYAN GHASEMNEZHAD, GIACINTO PAOLO (GP) SAGGESE, PAUL SMITH, HEANH SOK, AND VEDANSHUBHAI JOSHI

Abstract. Multi-tenant SaaS products often develop a small, expensive lie. The backend knows what a user may do; the frontend guesses what the user should see. Those guesses drift as tenants, roles, and modules change. The result is familiar to anyone who has operated a real platform: buttons that end in HTTP 403, pages that load and then deny access, support tickets that trace back to stale conditionals, and a product surface that quietly stops matching the authorization model.

We present UI-as-Policy, a production-tested pattern that treats the UI as a first-class policy surface. The backend materializes a tenant- and role-scoped UI contract at runtime. The web tier derives navigation, module visibility, layout, and client-side ability rules from that contract, while backend APIs continue to enforce authorization independently. Policy lives as data, so tenant onboarding and module changes become reviewed configuration updates rather than redeploys.

The paper contributes a name for the failure mode, a Truthful UX invariant for policy-gated surfaces, and an implementation pattern built around UI-as-Data, server-built CASL abilities, and auditable support impersonation. We report production measurements from Causify Platform: seven modules defined in the policy store, 25+ permission subjects, 150+ layout fields, zero-deployment tenant onboarding, 73–100% policy coverage on high-value UI surfaces, a 91% ETag cache-hit rate for identity revalidation, and 856 ms P50 full-materialization latency for the `/v2/me` control-plane contract.

Keywords. multi-tenant SaaS; authorization; UI policy; server-driven UI; impersonation; authorization drift; policy-as-data

Contents

Date: July 2026.

1. Introduction

A multi-tenant product makes promises. Every route, button, module, and sidebar entry says something to the user. It says: this belongs to you; this will work; this is part of your world.

Those promises are easy to break. The backend may enforce authorization correctly, while the frontend carries a separate permission model made from route guards, role checks, feature flags, and tenant-specific conditionals. The two models rarely fail in one dramatic incident. They drift one small decision at a time. A restricted tenant sees a billing module. A support engineer clicks into a page that was never enabled for that role. A module fetches configuration, discovers the user cannot use it, and renders a blank shell.

That is not only ugly product behavior. It is authorization drift. The UI is now describing a world the backend will not honor.

We adopt a blunt rule.

If the UI can lie about access, it is part of the security boundary.

UI-as-Policy is the discipline we use to make that boundary honest. The backend remains the enforcement point. The UI does not become trusted security code. Instead, the UI is rendered from a server-materialized contract that comes from the same policy inputs used for authorization. The frontend stops guessing.

The core primitive is UI-as-Data: a tenant- and role-scoped UI contract containing enabled modules, layout fragments, and the effective support context. The web tier consumes that contract to compose navigation and module layouts. It also builds the client-side ability model from a server-side profile, then exports packed rules for user-experience decisions. Backend APIs continue to authorize every request.

Concrete failure. During onboarding for a restricted tenant, a frontend conditional for a billing module was not updated with the backend entitlement record. The sidebar showed the billing link. The backend returned HTTP 403. Three support escalations later, the discrepancy traced back to a stale if-block. Under UI-as-Policy, the sidebar is rendered from the server-materialized module list; there is no standalone billing conditional to forget.

Contributions. This paper makes four contributions.

- It names the representation gap between backend enforcement and frontend rendering, then states a Truthful UX invariant for policy-gated surfaces.
- It describes UI-as-Data, a production pattern where tenant modules and layout fragments are materialized as a runtime UI contract instead of being scattered through frontend code.
- It shows how server-built CASL abilities, module layout checks, and backend enforcement keep the client useful without making it trusted.
- It presents a view-as support workflow with immutable actor attribution, server-enforced allow-lists, signed effective-context headers, and searchable audit logs.

The implementation uses Next.js, Cognito, AWS Lambda, DynamoDB, iron-session, and CASL [3, 2, 6, 4, 19, 8]. The pattern itself is not tied to those choices.

2. Authorization Drift

Most SaaS platforms start with a clean idea. The backend enforces access. The frontend checks enough state to avoid showing nonsense. That works while the product is small.

Then tenants diverge. One customer buys Grid, another buys Horizon, a demo tenant gets DataMap, and support engineers need temporary visibility into each of them. A few flags become a few route guards. A layout component grows tenant-specific branches. A module entry appears in one sidebar but not another. The backend remains the final authority, yet the UI has become a second policy system without the same discipline.

We call the gap between these two models the *representation gap*. It is the distance between what the backend will permit and what the UI presents as available.

2.1. **Requirements.** A multi-tenant UI policy system must satisfy five requirements.

R1. No stale gates.:
Policy-gated UI elements shown to a user must correspond to backend operations the system will actually permit for the user's current tenant and role.

R2. Configuration over redeploy.:
Enabling a tenant or changing a module entitlement should be a reviewed data change, not a code release.

R3. Safe impersonation.:
Support engineers need the tenant's perspective without shared credentials and without bypassing the authorization model.

R4. Defense in depth.:
The UI must never be the only barrier. Backend APIs authorize requests even when a browser is tampered with.

R5. Support at scale.:
Operators need scoped concurrent sessions and searchable audit trails, especially when several engineers are debugging different tenants at the same time.

Traditional RBAC handles a useful slice of this problem [18]. ABAC gives a richer model for attribute-derived decisions [11]. Feature flags help with release control [10]. Policy engines such as OPA, Cedar, and OpenFGA centralize authorization decisions [13, 9, 5, 15].

None of those choices automatically tells the frontend what world to render. They answer what is allowed. They do not prescribe how permission decisions become navigation, layout, module configuration, and support workflow. That missing layer is where UI-as-Policy sits.

Need	RBAC / ABAC	Feature flags	UI-as-Policy
Backend authorization	✓	×	✓
Tenant-level variability	Partial	✓	✓
UI derived from policy	×	Partial	✓
Config-only tenant changes	Partial	✓	✓
Auditable view-as support	×	×	✓
Defense against stale UI gates	×	×	✓

TABLE 1. Where UI-as-Policy fits. It does not replace authorization frameworks. It uses their decisions to make the rendered interface truthful.

2.2. **The Truthful UX invariant.** For policy-gated surfaces, we use the following invariant.

A rendered UI affordance must be backed by a server-side policy decision in the same effective tenant and role context.

The invariant is intentionally narrow. It does not claim every pixel on the page is authorization-sensitive. A help link or account menu may be universal. It does claim that navigation, modules, layouts, and policy-gated actions should not be rendered from a parallel frontend guess.

3. System Overview

UI-as-Policy has one moving part that matters more than the rest: the backend materializes a UI contract before the web tier renders the product. The contract is scoped to the authenticated actor, the effective tenant, and the effective role. The browser receives a product surface that has already been shaped by policy.

3.1. **Request lifecycle.** Figure 1 shows the request lifecycle.

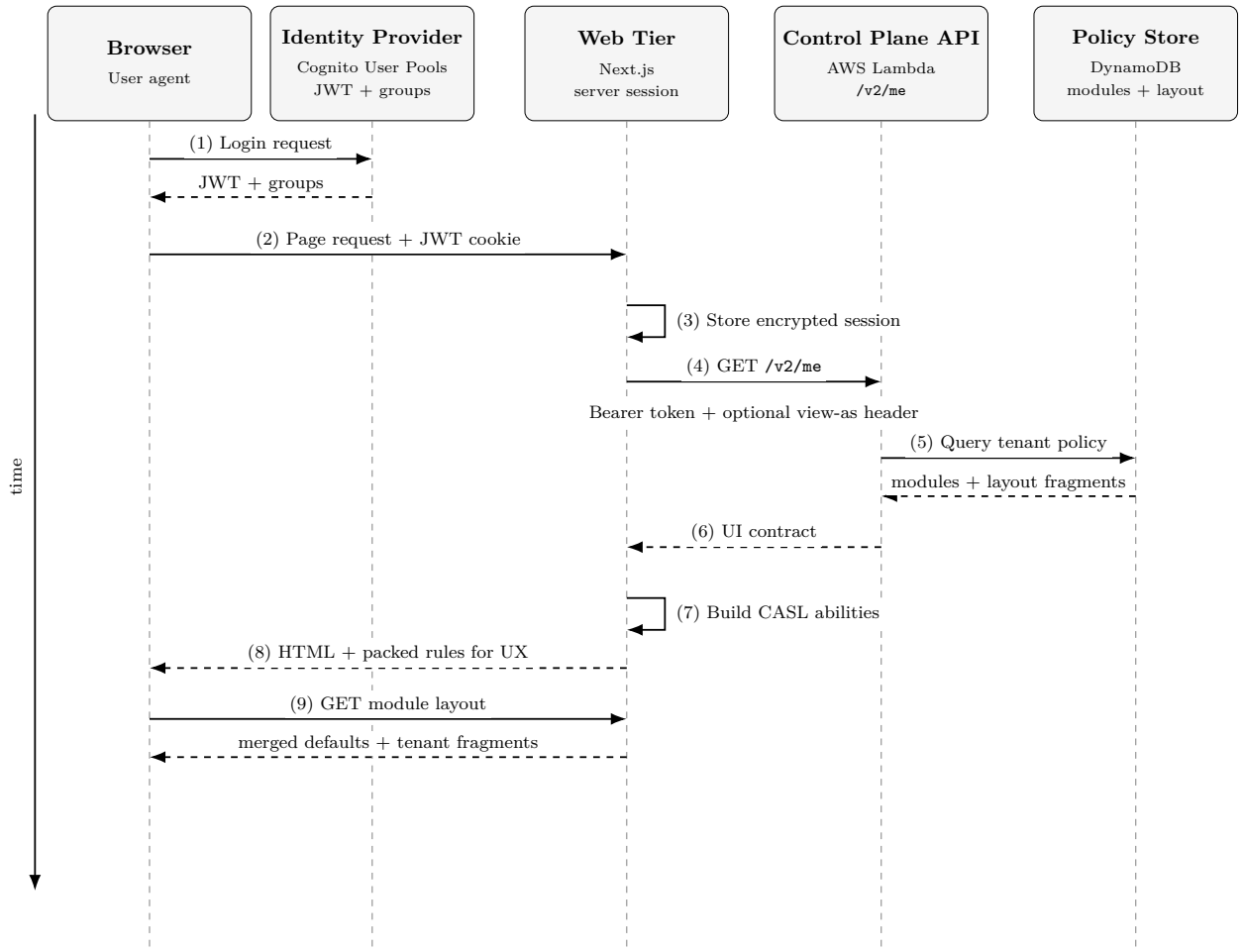

FIGURE 1. Production request lifecycle. The web tier calls `/v2/me`, receives a materialized UI contract, builds server-side abilities, and renders the client with packed rules for UX. Module layouts are lazy-loaded and checked server-side before they are returned.

The key response is step 6. `/v2/me` is not just a profile endpoint. It is the control-plane contract that tells the web tier which tenant world it should render.

3.2. **The UI contract.** A simplified response looks like this.

```json
{
  "actor": {
    "sub": "u123",
    "email": "engineer@example.com"
  },
  "view_as": {
    "tenant_id": "acme",
    "role": "admin",
    "modules": ["horizon", "grid"],
    "module_layout": { "sidebar": "full" }
  },
  "impersonatable": [
    {"tenant_id": "acme", "roles": ["member", "admin"]},
    {"tenant_id": "globex", "roles": ["member"]}
  ]
}
```

Listing 1. Representative `/v2/me` response for a two-module tenant.

The fields separate two identities that are often blurred.

Field	Meaning	Why it matters
`actor`	The real authenticated identity.	The audit trail always points to the human or service account that made the request.
`view_as`	The effective tenant and role context.	Support workflows can adopt a tenant perspective without erasing the actor.
`modules`	Enabled modules for the effective context.	Navigation and module entry points come from policy data.
`module_layout`	Structured layout fragments and defaults.	Tenant-specific UI shape travels with policy instead of frontend conditionals.
`impersonatable`	Server-computed allow-list of view-as targets.	The UI can offer only the targets the backend will accept.

Table 2. Core fields in the materialized UI contract.

3.3. **Module structure.** The platform defines seven independently evolvable modules: Horizon, Grid, Optima, Panorama, Frontier, Sentinel, and DataMap. Each module follows the same shape.

- A server layout checks the effective ability before rendering.
- A client module renders the experience selected by the contract.
- API routes inherit the same effective context from the server session.
- Module-specific layout is fetched only after a server-side ability check.

That last point is important. Layout is treated as privileged data. If a user cannot view a module, the server will not return its layout even if a modified browser requests it directly.

4. Policy Model

The policy model has two layers. The first decides which modules exist for a tenant and role. The second decides which features and actions are available inside those modules.

4.1. **Modules and layout as data.** Module access is stored per tenant and role in DynamoDB. A policy record contains an enabled-module list and a `moduleLayout` object with structured UI fragments. The control plane reads this record during `/v2/me` materialization and merges tenant fragments with shipped defaults.

Tenant changes are therefore small and reviewable. Enabling DataMap for a pilot tenant or switching a sidebar mode is a data change. Adding a new module type still requires code, since a new module brings routes, components, handlers, and documentation. That is the right boundary: routine tenant variability belongs in data; new product surface belongs in code review.

Malformed configuration is rejected during materialization. Partial configuration is allowed only when it can be safely merged with defaults. A tenant can override one sidebar group without redefining the whole navigation model.

4.2. **Server-built abilities.** Fine-grained permissions are expressed with CASL, an isomorphic JavaScript authorization library that serializes ability rules between server and client [8]. In our deployment the ability model covers three actions, `view`, `manage`, and `use`, across 25+ subjects. Those subjects include modules, generic pages, feature permissions, sub-module subjects, and config-derived subjects such as `module:config:integration:enabled`.

The server builds the ability object from the materialized profile. The client receives packed rules and uses them only for UX decisions: hiding buttons, disabling menu entries, and suppressing unavailable panels. Backend APIs still enforce authorization independently. A browser that tampers with packed rules may fool itself; it does not gain backend access.

4.3. **No hidden second policy language.** The trap in many products is accidental policy language. A frontend branch like `if (tenant === "acme")` looks harmless. Ten of them become an undocumented entitlement system.

UI-as-Policy draws a hard line. Frontend code may render from a contract. It may not invent access. The contract may be generated by RBAC, ABAC, a policy engine, a module registry, or a mix of those. What matters is that the rendered UI consumes the materialized decision instead of re-creating it.

Policy surface	**Scale**	**Runtime effect**
Modules in policy store	7	Drives navigation, dashboard cards, and module route access.
CASL permission subjects	25+	Controls module, feature, page, sub-module, and config-derived access.
`moduleLayout` fields	150+	Shapes sidebar mode, module defaults, settings tabs, feedback surfaces, and per-module fragments.
Sidebar modes	6	Lets tenants vary navigation behavior without code forks.
Tenant onboardings requiring deployment	0	Normal onboarding is a policy-store update plus identity assignment.

TABLE 3. Policy surface in the production deployment. Counts are taken from the deployed UI and control-plane configuration.

5. RUNTIME COMPOSITION

The UI is assembled in two steps. First, the server renders the shell from the `/v2/me` contract. Then module-specific layout is fetched on demand through a server route that repeats the ability check.

5.1. **Navigation from modules.** The sidebar does not know which tenant bought which product. It receives a module list. A single-module tenant sees one product surface. A two-module tenant sees two. A support engineer in view-as mode sees the module set of the effective tenant, not the broader support account.

This sounds almost too simple, which is the point. The system should not need a code search to answer why a sidebar entry appeared.

5.2. **Lazy layout loading.** Module layout is loaded through a dedicated endpoint. Before returning layout, the server checks whether the effective context can view the requested module. Disabled modules therefore fail closed twice: the navigation never links to them, and direct layout fetches return HTTP 403.

Tenant fragments are deep-merged over shipped defaults. That gives operators useful flexibility without handing them the whole product definition. A tenant can change a dashboard default, sidebar mode, or integration panel while stable module code still owns the implementation.

5.3. **CI door check.** The Truthful UX invariant can be tested per release. For each tenant and role in a test matrix, the check is straightforward.

(1) Fetch `/v2/me` and extract the enabled module set M.
(2) Assert that `/api/auth/user/module-layout?module=X` returns HTTP 200 for $X \in M$ and HTTP 403 for $X \notin M$.
(3) Assert that routes belonging to M render successfully, while routes outside M reject or redirect.
(4) Assert that the packed CASL rules are derived from the same server profile used to build the page.

The check is not a replacement for backend authorization tests. It catches a different problem: a frontend that has started to tell a different story from the backend.

6. Safe View-As Without Shared Accounts

Support engineers often need to see a tenant's UI exactly as that tenant sees it. The lazy answer is a shared admin account. It works until it matters. Then the audit trail points to a credential instead of a person, the blast radius is concentrated in one login, and every incident review starts with a shrug.

Causify Platform uses a view-as workflow instead. The actor stays fixed. The effective context changes.

6.1. **Actor and effective context.** The `actor` field in `/v2/me` is the authenticated identity. It does not change during a session. The `view_as` field is the adopted tenant and role. Every request made during a support session carries both values in logs, so the system can answer two different questions.

- Who actually clicked the button?
- Which tenant and role were they viewing when they clicked it?

Those questions should never share one field.

6.2. **Server-enforced allow-lists.** The control plane computes the `impersonatable` list from the actor's role and support scope. The web tier may display this list, but it does not own it. When a support engineer activates view-as, the control plane validates the requested tenant and role against the server-computed allow-list before applying the effective context.

A modified browser can ask for another tenant. The middleware still says no.

6.3. **Signed view-as headers.** The web tier signs the effective context with HMAC-SHA256. The signed payload contains the tenant, role, and a per-activation nonce. The signing key stays server-side; the browser never receives it. The nonce is registered at activation and revoked on logout or tenant switch. If nonce registration fails, activation fails closed.

```
const nonce = randomUUID();
const viewAs = `tenant=${tenantId};role=${role};nonce=${nonce}`;
const signature = signWithServerKey(viewAs);
```

Listing 2. Sanitized view-as activation sketch.

The signature is not the only control. API Gateway still validates the Cognito token, the Lambda still checks the allow-list, backend APIs still enforce authorization, and logs still contain immutable actor attribution. The signing layer narrows replay and header-injection risk; it does not carry the whole security model.

7. Implementation Notes

The deployed system uses AWS-managed identity and data services, a server-rendered web tier, and a modular React application. The details below are not requirements for the pattern, but they show where the sharp edges live.

7.1. **Control plane.** The control plane is an AWS Lambda API behind API Gateway. Middleware validates Cognito JWTs, resolves tenant membership from the policy store, computes effective role and module access, applies view-as guardrails, and returns the `/v2/me` contract. AWS Lambda Powertools provides the middleware structure and observability hooks [7].

The endpoint uses ETag and `If-None-Match` semantics so repeated identity checks do not force full materialization. Full materialization still happens when the identity fingerprint changes, when the session changes, or when a page load needs a fresh contract.

7.2. **Web tier.** The web tier stores session state with iron-session [19]. Server components read the session, call the control plane, build CASL abilities, and render the application shell. Packed CASL rules are sent to the client for UX consistency. They are treated as hints, never authority.

7.3. **Policy store governance.** Moving UI policy into DynamoDB changes the operational surface. A buggy write can alter the UI contract for an entire tenant. That is better than sprinkling tenant logic through a codebase, but it is not free.

In production, writes are IAM-scoped, reviewed, and parsed strictly at materialization. Future work should add versioned JSON Schema validation and CI checks before policy records reach production. Policy-as-data still needs governance; it simply makes the governance visible.

7.4. **Caching trade-off.** A policy edit takes effect at the next full materialization or page load. That is acceptable for tenant onboarding and module toggles, which are operator actions. For urgent security changes, push invalidation or policy-version bumps would shorten the window. We discuss this in Section 10.

8. Evaluation

We evaluate UI-as-Policy along five dimensions: truthfulness, operational leverage, policy depth, runtime cost, and defense in depth. The results are from one production deployment of Causify Platform. They should be read as an experience report, not a controlled experiment.

8.1. **Threat model.** UI-as-Policy addresses stale UI gates, impersonation abuse, and client-side authorization bypass. It does not attempt to replace standard cloud controls for secrets, network isolation, IAM, or database security.

The policy store is the new critical surface. Moving policy from code to DynamoDB shifts the blast radius of a bad write from a code branch to the UI contract of a tenant. The mitigation is not denial. Operators must treat policy-store writes with the same seriousness as code deploys.

8.2. **Truthful UX coverage.** Table 4 summarizes the production policy surface and coverage. Coverage means that a surface conditionally renders based on at least one CASL ability check, module membership test, or module-layout field.

Metric	**Value**
Modules defined in policy store	7
CASL permission subjects	25+
`moduleLayout` config fields	150+
Tenant onboardings requiring deployment	0
Policy surface coverage	
Navigation items	8/11 (73%)
Module components	7/7 (100%)
Configuration tabs	5/6 (83%)
UI elements hidden for restricted tenants	
Single-module tenant	6/7 (86%)
Two-module tenant	5/7 (71%)

TABLE 4. Policy adoption and coverage in production. The remaining navigation entries are generic surfaces such as profile, documentation, and global account actions.

High-value surfaces are covered. Module components are fully policy-driven. Configuration tabs are almost fully covered. Navigation still has universal entries, which is expected. Those entries are intentionally shared across tenants and roles.

Before adoption, permission and navigation mismatches produced recurring support escalations that needed developer investigation. Since adoption, we have observed no drift incidents against policy-gated surfaces. This is not a universal claim about all authorization bugs. It is a narrower result: the architecture eliminated the stale-gate class for surfaces that were migrated to the contract.

8.3. **Runtime cost.** Table 5 reports full-materialization latency for `/v2/me`. The path includes JWT validation, tenant and role reads, layout-fragment assembly, impersonation-list assembly, and CASL rule construction.

Metric	**512 MB Lambda (ms)**
P50	856
P75	938
P90	1,038
P95	1,112
P99	1,176

TABLE 5. Controller Lambda full-materialization latency from 1,368 production observations, March–April 2026, 512 MB configuration.

Full materialization is not the steady-state path. ETag-conditional revalidations dominate normal traffic at a 91% cache-hit rate. Those return HTTP 304 with a P50 of 68 ms, with median user-visible revalidation latency of roughly 75 ms.

8.4. **Operational leverage.** The production deployment serves more than ten tenants, each with multiple users across roles. Adding a tenant requires a policy-store record and Cognito group assignment. Enabling a module or changing layout is a policy update. No code deployment is required for routine tenant onboarding.

The operational gain is mundane and valuable. A tenant change stops being a front-end release. Support can reason from one contract. QA can fetch `/v2/me` and inspect exactly which product world the user will see.

8.5. **Defense in depth.** Six layers protect the path between browser and backend.

Layer	Where enforced	Mechanism
Edge middleware	Server	Session check, path validation, and security headers.
Server components	Server	Redirect on missing or invalid session.
CASL abilities	Server + client	Built server-side from `/v2/me`; exported for UX only.
Module gating	Server	`can('view', module)` before layout is returned.
View-as signing	Server	HMAC-signed effective context; secret never reaches the browser.
Control plane	Server	JWT validation, allow-list membership, and schema-prefix derivation.

TABLE 6. Defense-in-depth layers. The client participates in UX decisions, but backend and control-plane checks remain authoritative.

A broken client-side rule should at worst produce a confusing interface. It should not produce a permitted backend response.

8.6. **What the evaluation does not show.** The evaluation is correctness-oriented. It shows that covered surfaces derive from a server contract and that the deployed system carries useful production metrics. It does not include a controlled before/after incident study, nor does it prove absence of drift in legacy surfaces. Longitudinal drift measurement and a full tenant-matrix CI gate remain future work.

9. Related Work

RBAC, ABAC, and policy formalisms. Role-based access control formalizes role-to-permission assignment [18]. ABAC derives decisions from attributes rather than static roles [11]. XACML standardized the policy-enforcement-point and policy-decision-point split [12]. UI-as-Policy is compatible with these models. Its concern is the next step: how an authorization decision becomes a rendered product surface.

Feature flags. Feature flags separate release from deployment [10]. They are useful, but they can become a parallel entitlement system. UI-as-Policy borrows the operational convenience of configuration while binding it to tenant and role semantics.

Policy engines. OPA, Cedar, Amazon Verified Permissions, and OpenFGA provide ways to centralize policy logic [13, 9, 5, 15]. These systems can feed UI-as-Policy. They do not, by themselves, make a sidebar, module layout, or settings page truthful.

Relationship-based authorization. Zanzibar describes Google's global authorization system for relationship-based permissions [17]. It shows how authorization becomes a distributed-systems problem at scale. UI-as-Policy deals with a nearby problem inside product code: keeping the UI aligned with whatever authorization system the backend trusts.

Server-driven UI.. Server-driven UI architectures return layout and interaction structure from the server [1]. UI-as-Policy is narrower and stricter. It uses a server contract to shape the UI, and it ties that contract to authorization inputs.

Impersonation risk. OWASP calls out broken access control as a top web security risk and flags account impersonation as a dangerous operational shortcut [16, 14]. The view-as design in this paper follows the safer pattern: keep actor attribution immutable, validate effective context server-side, and log both identities.

10. Limitations and Future Work

UI-as-Policy trades hidden frontend logic for explicit policy data. That is a better place for the complexity, not a magic deletion of it.

Schema governance. The current system rejects malformed layout at materialization time. A stronger version would use versioned JSON Schema, offline validation, and migration tooling before records reach production.

Propagation delay. Policy changes currently take effect at the next page load or full materialization. DynamoDB Streams, policy-version bumps, or push invalidation would shorten the window for urgent changes.

View-as verification. The signing flow narrows replay and header-injection risk. A future hardening step is end-to-end signature verification in the control plane, with key identifiers and rotation metadata logged alongside activation events.

Coverage beyond migrated surfaces. The invariant holds for surfaces migrated to the contract. Legacy components still need audit and migration. A full tenant-matrix CI gate should become a release door, not a best-effort test.

Authoring experience. A policy-as-data system deserves an operator interface: preview, diff, approval, rollback, and tenant-scoped simulation. Without that, the policy store risks becoming the new place where tribal knowledge hides.

11. Conclusion

Multi-tenant SaaS often fails in a very human way. The backend tells the truth. The UI tells a hopeful story. Users then discover the truth one failed click at a time.

UI-as-Policy replaces hope with a contract. A tenant and role receive a materialized UI contract. The web tier renders from it. Abilities are built server-side and exported only for UX. Backend APIs remain authoritative. Support engineers can view the tenant world without sharing credentials or erasing the actor identity.

The pattern is not exotic. That is part of its appeal. It is a disciplined arrangement of familiar pieces: identity, policy data, server rendering, ability rules, module gating, and audit logs. The value comes from refusing to let the frontend maintain its own little mythology of who can do what.

> **The sidebar is a security boundary when users rely on it to understand what they are allowed to do.**

Once a team accepts that, UI truthfulness becomes an engineering property instead of a product accident.

References

[1] Airbnb Engineering. A deep dive into airbnb's server-driven ui system. Engineering blog. Accessed 2026-07-15. URL: `https://medium.com/airbnb-engineering/`.

[2] Amazon Web Services. Adding groups to a user pool. Amazon Cognito Documentation. Accessed 2026-07-15. URL: `https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-user-groups.html`.

[3] Amazon Web Services. Amazon cognito user pools. Documentation. Accessed 2026-07-15. URL: `https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools.html`.

[4] Amazon Web Services. Amazon dynamodb developer guide. Documentation. Accessed 2026-07-15. URL: https://docs.aws.amazon.com/dynamodb/.
[5] Amazon Web Services. Amazon verified permissions user guide. Documentation. Accessed 2026-07-15. URL: https://docs.aws.amazon.com/verifiedpermissions/.
[6] Amazon Web Services. Aws lambda developer guide. Documentation. Accessed 2026-07-15. URL: https://docs.aws.amazon.com/lambda/.
[7] AWS Lambda Powertools. Powertools for aws lambda (python). Documentation. Accessed 2026-07-15. URL: https://docs.aws.amazon.com/powertools/python/latest/.
[8] CASL Authors. Casl: Isomorphic authorization javascript library. Documentation. Accessed 2026-07-15. URL: https://casl.js.org/.
[9] Cedar Policy Language Authors. Cedar policy language reference guide. Documentation. Accessed 2026-07-15. URL: https://docs.cedarpolicy.com/.
[10] Martin Fowler. Feature toggles (aka feature flags). Website, 2010. Accessed 2026-07-15. URL: https://martinfowler.com/articles/feature-toggles.html.
[11] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (abac) definition and considerations. Technical Report Special Publication 800-162, National Institute of Standards and Technology, 2014. URL: https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-162.pdf, doi:10.6028/NIST.SP.800-162.
[12] OASIS. extensible access control markup language (xacml) version 3.0. OASIS Standard, 2013. URL: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html.
[13] Open Policy Agent Contributors. Open policy agent documentation. Documentation. Accessed 2026-07-15. URL: https://www.openpolicyagent.org/docs/.
[14] Open Web Application Security Project. Cd-sec-02: Account impersonation. OWASP Citizen Development Top 10 Security Risks, 2022. Accessed 2026-07-15. URL: https://owasp.org/www-project-citizen-development-top10-security-risks/content/2022/en/CD-SEC-02-Account-Impersonation.
[15] OpenFGA Contributors. Openfga documentation. Documentation. Accessed 2026-07-15. URL: https://openfga.dev/docs/.
[16] OWASP Foundation. A01:2021 broken access control. OWASP Top 10, 2021. Accessed 2026-07-15. URL: https://owasp.org/Top10/A01_2021-Broken_Access_Control/.
[17] Ruoming Pang, Manuel Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey Shulman, Mike Tarjan, Gang Wang, and Calum Wright. Zanzibar: Google's consistent, global authorization system. In *Proceedings of the 2019 USENIX Annual Technical Conference*, 2019. URL: https://www.usenix.org/system/files/atc19-pang.pdf.
[18] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. In *IEEE Computer*, volume 29, pages 38–47, 1996. doi:10.1109/2.485845.
[19] vvo and contributors. iron-session. GitHub repository. Accessed 2026-07-15. URL: https://github.com/vvo/iron-session.

About the Authors

Shayan Ghasemnezhad is Infrastructure and DevOps Lead at Causify AI, Turin, Italy. His research interests include platform engineering and cloud solutions architecture, production AI/ML systems, and secure, reliable infrastructure operations. Ghasemnezhad received his Bachelor's degree in Digital Management from Ca' Foscari University of Venice. Contact him at shayan@causify.ai.

Giacinto Paolo (GP) Saggese is Co-founder and CTO at Causify AI, Potomac, Maryland, USA. His research interests include machine learning, fintech, and the commercialization of research. Saggese received his Ph.D. degree in Electrical and Computer Engineering from the University of Illinois Urbana-Champaign. He is an Adjunct Professor at the University of Maryland, College Park. Contact him at gp@causify.ai.

Paul Smith is Co-founder and Chief Scientist at Causify AI, Cary, North Carolina, USA. His research interests include causal inference, Bayesian modeling, and time-series prediction. Smith received his Ph.D. degree in Mathematics from UCLA. Contact him at paul@causify.ai.

Heanh Sok is Software Engineer at Causify AI, College Park, Maryland, USA. His research interests include big data systems, distributed systems, and AI/ML engineering and operations. Sok received

his Master's degree in Data Science from the University of Maryland, College Park. Contact him at h.sok@causify.ai.

Vedanshubhai Joshi is Software Engineer at Causify AI, Indiana, USA. His research interests include platform infrastructure, multi-tenant API design, and data engineering pipelines. Joshi received his Master's degree in Computer Science from Purdue University. Contact him at v.joshi@causify.ai.